Node.js Backend Development — Lecture 8

Detailed English Notes | CRUD APIs, URL Parameters, Express Router, Update and Delete

Basis: These notes follow the uploaded lecture transcript. Code snippets are clean illustrative versions of the patterns demonstrated in the lecture; adjust file names and schema fields to match your project.

1. What this lecture covers

- Reviewing the MenuItem POST and GET endpoints from the previous lecture.
- Planning API endpoints before implementation using a CRUD diagram.
- Parameterized URLs and filtering people by their work/role.
- Organizing endpoints with Express Router instead of keeping everything in server.js.
- Updating a document using its MongoDB _id with PUT and findByIdAndUpdate().
- Deleting a document using its _id with DELETE and findByIdAndRemove()/the equivalent supported by your Mongoose version.
- Testing requests in Postman and interpreting HTTP status codes.
- Homework: create the equivalent parameterized, update, and delete endpoints for MenuItem based on its schema.

2. Recap: MenuItem POST and GET

POST /menu

The client sends a JSON object in the request body. Express JSON middleware parses it, the route creates a new MenuItem document, and the document is saved to MongoDB. Because saving is asynchronous, the route uses async/await.

```
router.post("/", async (req, res) => {
  try {
    const newItem = new MenuItem(req.body);
    const savedItem = await newItem.save();
    res.status(201).json(savedItem);
  } catch (err) {
    console.error(err);
    res.status(500).json({ error: "Internal server error" });
  }
});
```

GET /menu

The route retrieves the menu items from MongoDB and sends them as JSON. The database operation may take time, so await is used.

```
router.get("/", async (req, res) => {
  try {
    const items = await MenuItem.find();
    res.status(200).json(items);
  } catch (err) {
    console.error(err);
    res.status(500).json({ error: "Internal server error" });
  }
});
```

The transcript demonstrates saving and fetching records. A successful create commonly uses HTTP 201 Created, while a successful read commonly uses 200 OK.

3. Plan APIs using CRUD

Before writing endpoints, teams commonly decide what the application must support and map the operations. The lecture uses a diagramming tool such as diagrams.net (draw.io) to visualize the API plan.

- **Create:** create a new record — usually POST.
- **Read:** retrieve records — usually GET.
- **Update:** change an existing record — often PUT or PATCH.
- **Delete:** remove an existing record — DELETE.

For the restaurant example, Person records can represent chefs, waiters, and managers; MenuItem records represent dishes. The lecture's diagram groups these operations around the relevant collection/resource.

Planning endpoints first makes the API easier to discuss, implement, test, and maintain. Do not create a separate endpoint for every possible role or taste when one parameterized route can handle the category.

4. Parameterized URLs

The problem with separate endpoints

Suppose GET /person returns every person, but the client wants only chefs, waiters, or managers. Creating a separate hard-coded endpoint for each role is repetitive and does not scale when new roles are added.

Use a route parameter

```
router.get("/work/:workType", async (req, res) => {
  try {
    const workType = req.params.workType;

    if (!["chef", "manager", "waiter"].includes(workType)) {
      return res.status(404).json({ error: "Invalid work type" });
    }

    const people = await Person.find({ work: workType });
    return res.status(200).json(people);
  } catch (err) {
    console.error(err);
    return res.status(500).json({ error: "Internal server error" });
  }
});
```

In the lecture, the route is described in the form **/person/:workType**. If the router is mounted at **/person**, the router-local path can be **/work/:workType** or another agreed path. Keep the final URL consistent with how the router is mounted.

How it works

- **Colon syntax:** :workType marks a variable segment in the URL.
- **req.params:** Express places route parameter values in this object.
- A request such as GET /person/work/chef provides the value "chef" in req.params.workType when the route is defined as /work/:workType under the /person mount.
- The query Person.find({ work: workType }) returns documents whose work field matches the provided role.
- Validation checks the supplied value before making a database query.

Why validate first?

A client can send unexpected values such as /person/work/dog. The lecture checks that the value is one of the allowed work types before querying the database. This avoids unnecessary database operations and returns a clear response for invalid input.

Use the actual allowed values from your schema. The transcript mentions chef, manager, and waiter, but your schema may use different capitalization or values. Match the enum exactly.

5. HTTP status codes discussed

- **200 OK:** the request succeeded, commonly used for successful GET, update, or delete responses when a body is returned.
- **201 Created:** a resource was successfully created.
- **404 Not Found:** the requested resource was not found, or the lecture's validation branch returns this for an invalid work type.
- **500 Internal Server Error:** an unexpected server/database error occurred.

Status codes help clients understand the outcome without relying only on the response message.

API design note: 400 Bad Request is often a better fit for an invalid parameter value, while 404 is a natural fit when a requested record does not exist. The lecture uses 404 for its invalid work-type example; know the lecture's version and understand the distinction.

6. Express Router: organize routes into separate files

Why not put every endpoint in server.js?

As the application grows, one server file can contain many POST, GET, PUT, and DELETE handlers for several resources. This becomes difficult to read and maintain. Express Router lets you group related endpoints in separate files.

- Create a routes folder.
- Create personRoutes.js for Person endpoints.
- Create menuItemRoutes.js for MenuItem endpoints.
- Define routes using router.get(), router.post(), router.put(), and router.delete().
- Export the router from each file and mount it in server.js.

Example: personRoutes.js

```
const express = require("express");
const router = express.Router();
const Person = require("../models/Person");

// Define Person routes here.
// Example: router.get("/", ...);
// Example: router.post("/", ...);

module.exports = router;
```

Example: menuItemRoutes.js

```
const express = require("express");
const router = express.Router();
const MenuItem = require("../models/MenuItem");

// Define MenuItem routes here.

module.exports = router;
```

Mount routers in server.js

```
const express = require("express");
const app = express();

app.use(express.json());

const personRoutes = require("./routes/personRoutes");
const menuItemRoutes = require("./routes/menuItemRoutes");

app.use("/person", personRoutes);
app.use("/menu", menuItemRoutes);
```

```
// Keep your existing database connection and app.listen(...) setup.
```

The first argument to app.use() is the common URL prefix. The router handles the remaining path. For example, router.get("/") mounted at /person becomes GET /person.

Common mistake highlighted by the lecture

After moving code into a router file, an error such as “Person is not defined” can occur because the model was not imported into that new file. Every module has its own scope. Import the model where it is used.

```
const Person = require("../models/Person");
```

The relative path depends on the actual folder structure. If routes and models are sibling folders, ../models/Person is a common path.

7. Updating a Person: PUT and document ID

Two pieces of information are needed

- Which record should change? Identify it using the document’s unique MongoDB _id.
- What should change? Send the updated fields in the JSON request body.

The lecture recommends sending the record identifier as a URL parameter and the fields to update in the request body.

Example endpoint

```
router.put("/:id", async (req, res) => {
  try {
    const personId = req.params.id;
    const updatedData = req.body;

    const updatedPerson = await Person.findByIdAndUpdate(
      personId,
      updatedData,
      { new: true, runValidators: true }
    );

    if (!updatedPerson) {
      return res.status(404).json({ error: "Person not found" });
    }

    return res.status(200).json(updatedPerson);
  } catch (err) {
    console.error(err);
    return res.status(500).json({ error: "Internal server error" });
  }
});
```

Understanding the code

- **router.put("/:id", ...):** defines a PUT endpoint with a variable ID.
- **req.params.id:** reads the identifier from the URL.
- **req.body:** contains the fields the client wants to update.
- **findByIdAndUpdate(id, data, options):** finds a document by its _id and updates it.
- **{ new: true }:** returns the updated document rather than the old document.
- **{ runValidators: true }:** asks Mongoose to run update validators.
- **if (!updatedPerson):** handles the case where no document with that ID exists.
- **try/catch:** handles unexpected errors, such as database or request-processing failures.

Postman test

- Choose PUT and use a URL such as http://localhost:3000/person/YOUR_OBJECT_ID.

- Select Body → raw → JSON.
- Send the field(s) you want to change, for example { "name": "Alice Sharma" }.
- Check the response and then GET the person list or inspect MongoDB to confirm the update.

The lecture calls out the options new: true and runValidators: true. The exact schema validation behavior can differ between document saves and update operations, so explicitly enabling update validators is useful.

8. MongoDB _id / ObjectId

MongoDB documents have a unique _id field by default. In a Mongoose-backed application, it is commonly represented as an ObjectId. The lecture refers to it as _id, ObjectId, or simply ID.

- Use the ID to identify the exact document to update or delete.
- The parameter name in the route is your choice: :id, :personId, or another clear name. The parameter name is not itself the MongoDB field name.
- The route parameter is read from req.params; it is not automatically the same thing as req.body.

Example: PUT /person/66f... identifies the record by URL and the body contains the new field values.

Malformed ObjectId values can throw a CastError in some Mongoose operations. The lecture's catch block returns a server error for errors generally; a production API can distinguish invalid IDs (400) from missing records (404).

9. Deleting a Person

Delete requires the identifier of the record to remove. It does not need an update body in the basic pattern shown in the lecture.

```
router.delete("/:id", async (req, res) => {
  try {
    const personId = req.params.id;

    const deletedPerson = await Person.findByIdAndDelete(personId);

    if (!deletedPerson) {
      return res.status(404).json({ error: "Person not found" });
    }

    return res.status(200).json({
      message: "Person deleted successfully",
      person: deletedPerson
    });
  } catch (err) {
    console.error(err);
    return res.status(500).json({ error: "Internal server error" });
  }
});
```

The transcript demonstrates a Mongoose helper described as findByIdAndRemove(). Depending on the Mongoose version, prefer the supported current equivalent findByIdAndDelete(). Both patterns express the idea: locate the document by ID and remove it.

Postman test

- Choose DELETE and use http://localhost:3000/person/YOUR_OBJECT_ID.
- Send the request.
- Confirm the success response and check that the record no longer appears in a subsequent GET request or database view.
- Try a non-existent ID to test the 404 branch.

10. CRUD route summary

Example route layout when the routers are mounted in server.js as shown above:

```
PERSON
POST   /person          Create a person
GET    /person          Fetch all people
GET    /person/work/:workType
                       Fetch people filtered by work type
PUT    /person/:id      Update a person by ID
DELETE /person/:id      Delete a person by ID

MENU ITEM
POST   /menu            Create a menu item
GET    /menu            Fetch all menu items
GET    /menu/taste/:taste
                       Homework: filter menu items by taste
PUT    /menu/:id        Homework: update a menu item by ID
DELETE /menu/:id        Homework: delete a menu item by ID
```

The exact route for the taste filter is an implementation choice; the lecture's homework describes a parameterized MenuItem route based on taste, such as sour, sweet, or spicy. Keep route paths consistent throughout the project.

11. Homework from the lecture

Complete the equivalent operations for MenuItem yourself.

- Build a parameterized GET endpoint that filters menu items by taste.
- Validate taste against the values supported by your MenuItem schema (the lecture mentions sour, sweet, and spicy).
- Create a PUT endpoint to update a MenuItem by its _id.
- Create a DELETE endpoint to delete a MenuItem by its _id.
- Keep MenuItem endpoints in menuItemRoutes.js and mount the router in server.js.
- Test success, invalid taste, unknown ID, and error cases using Postman.

The transcript does not provide the complete MenuItem schema field list. Use the schema file already provided in your project rather than inventing fields. Check the exact enum values for taste.

12. Debugging checklist

- "Person is not defined" → import the Person model in personRoutes.js.
- "MenuItem is not defined" → import the MenuItem model in menuItemRoutes.js.
- req.body is undefined → confirm app.use(express.json()) is registered before the routes and Postman sends raw JSON.
- Route not found → check the mounted prefix in server.js and the local path inside the router.
- Updated record is not returned → confirm the update options include new: true.
- Schema validators do not seem to run during update → include runValidators: true.
- Record not found → verify the ID belongs to the intended collection and check the 404 branch.
- Server error → inspect the terminal error; do not assume every failure means the record is missing.
- Router changes do not appear → save files and restart the server unless a watcher such as nodemon is running.

13. Quick revision questions

1. Why use a parameterized URL?
To handle many values with one route instead of creating a separate endpoint for each value.

2. What does :workType mean?
A named route parameter whose value is available as req.params.workType.

3. What is the difference between req.params and req.body?
req.params contains values from the URL path; req.body contains submitted request data, typically parsed JSON.

4. Why validate a route parameter before querying MongoDB?
To reject unsupported values early and avoid unnecessary database operations.

5. What is Express Router used for?
To group related routes into modules and keep server.js organized.

6. Why import Person in personRoutes.js?
Each JavaScript module has its own scope; a model imported in server.js is not automatically available in another file.

7. Why does an update need both an ID and a body?
The ID selects the document; the body specifies the fields to change.

8. What does new: true do in findByIdAndUpdate()?
It returns the updated document.

9. What does runValidators: true do?
It enables Mongoose update validators for the update operation.

10. What should happen if no document matches the ID?
Return a not-found response, commonly 404.

11. What is the purpose of try/catch in these async routes?
To catch rejected asynchronous operations and send a controlled error response.

12. What is the homework?
Build MenuItem filtering by taste, plus update and delete endpoints, and test them.

14. Final takeaway

This lecture moves from basic create/read routes toward a more maintainable REST-style backend. Learn the flow: define the route → read URL parameters/body → validate inputs → await the Mongoose operation → handle a missing record → send a suitable status code and JSON response. Express Router keeps resource-specific code organized, while MongoDB's _id lets update and delete operations target one exact document.

End of Lecture 8 notes.